



UNIDAD 3

Métodos en conjunto y ajuste de modelos.

TEMA 1:

Modelos de ensamble.

Descripción:

La Unidad 3 explora dos ejes fundamentales del aprendizaje automático: los métodos en conjunto y el ajuste de hiperparámetros.

Los primeros se enfocan en combinar múltiples modelos para mejorar la precisión, reducir el sesgo o la varianza, y aumentar la capacidad de generalización.

Aquí se estudian enfoques como bagging, random forests, boosting y stacking, ampliamente aplicados en problemas reales.

El segundo eje profundiza en técnicas para optimizar modelos, desde Grid Search y Random Search hasta la optimización bayesiana y el uso de pipelines, consolidando la capacidad del estudiante para seleccionar y ajustar modelos de manera sistemática y eficiente.

Machine Learning 1

PERIODO ACADÉMICO 2026-1S

TABLA DE CONTENIDO

Tema 1. Modelos de ensamble.....	2
1.1. Bagging, pasting y out-of-bag evaluation.....	2
1.2. Bosques Aleatorios (Random Forests) y Árboles Extremadamente Aleatorizados (Extra-Trees).....	5
1.3. Métodos de Potenciación (Boosting: AdaBoost, Gradient Boosting, XGBoost).....	8
1.4. Stacking y Blending de Modelos.....	12
Bibliografía	16

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

Tema 1. Modelos de ensamble.

Los **modelos de ensamble** parten de una idea simple y poderosa: combinar múltiples modelos base para obtener un predictor más estable, preciso y generalizable que cualquiera de sus componentes por separado. Esta familia incluye enfoques que reducen **varianza** mediante muestreo y agregación (p. ej., **bagging** y **pasting**), donde la estimación **out-of-bag (OOB)** ofrece una evaluación interna del error sin recurrir a un conjunto de validación adicional. Sobre esta base se construyen métodos ampliamente usados como los **Bosques Aleatorios (Random Forests)** y los **Extra-Trees**, que fomentan la diversidad al seleccionar características (y, en Extra-Trees, umbrales) de forma aleatoria, logrando modelos robustos y eficientes en datos tabulares (Müller & Guido, 2022; Amini & Bach, 2021).

En paralelo, los métodos de **potenciación (boosting)** —**AdaBoost**, **Gradient Boosting** y **XGBoost**— optimizan secuencialmente los errores residuales, ofreciendo alto rendimiento a costa de mayor sensibilidad a la configuración de **hiperparámetros** y al riesgo de sobreajuste si no se regularizan adecuadamente (Chen & Guestrin, 2016). Finalmente, **stacking** y **blending** aprenden a **combinar** predicciones heterogéneas con un **meta-modelo**, materializando el principio de que ningún algoritmo es universalmente superior (Wolpert, 1992).

Pedagógicamente, los ensambles permiten observar con claridad el compromiso **sesgo-varianza**, medir el impacto de la **diversidad** entre modelos y practicar evaluaciones rigurosas (OOB, validación cruzada). En Colab, su implementación guiada facilita contrastar enfoques y justificar, con evidencia, la **selección del mejor modelo** para cada problema.

1.1. Bagging, pasting y out-of-bag evaluation.

Los modelos de ensamble han revolucionado el aprendizaje automático al demostrar que la combinación de múltiples clasificadores débiles puede producir un modelo significativamente más robusto que cualquiera de sus componentes individuales.

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

Bagging (Bootstrap Aggregating) y pasting son dos variantes que parten de la misma idea: entrenar varios modelos sobre subconjuntos de datos y combinar sus predicciones. La diferencia esencial radica en cómo se generan estos subconjuntos: **bagging** selecciona las muestras con reemplazo, mientras que **pasting** las selecciona sin reemplazo (Müller & Guido, 2022).

En la práctica, bagging introduce variabilidad y permite reducir la varianza de modelos como los árboles de decisión, mientras que pasting es útil cuando se busca aprovechar la totalidad de las observaciones sin repeticiones. Un componente esencial en este contexto es la **evaluación out-of-bag (OOB)**, que aprovecha las instancias no seleccionadas en el muestreo bootstrap para estimar el error de generalización sin necesidad de un conjunto de validación adicional (Amini & Bach, 2021).

Desde el punto de vista pedagógico, es importante destacar que bagging permite comprender la noción de estabilidad en el aprendizaje automático: si un modelo tiende a sobreajustarse, su combinación con otros mediante ensamble reduce ese riesgo. De esta manera, los estudiantes pueden ver que la fortaleza de un bosque está en la diversidad de sus árboles.

Figura 1. Métodos de muestreo para el entrenamiento de algoritmos



Fuente: Elaboración propia

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

Para ilustrar el funcionamiento de la técnica **Bagging** y el uso de la evaluación *out-of-bag* (OOB), se presenta el siguiente fragmento de código en Python utilizando la librería *scikit-learn*.

En este ejemplo, se generan datos sintéticos mediante `make_classification` con 20 atributos, de los cuales 15 son informativos y 5 redundantes, lo que permite simular un problema de clasificación binaria complejo.

El clasificador base seleccionado es un árbol de decisión (`DecisionTreeClassifier`), sobre el cual se construye un **BaggingClassifier** con 50 estimadores.

Se activa la opción `oob_score=True`, que permite calcular la precisión del modelo a partir de las observaciones no incluidas en las muestras de entrenamiento de cada árbol.

Finalmente, se muestran en pantalla tanto la **precisión en el conjunto de prueba** como la **precisión OOB**, evidenciando la capacidad del método para evaluar su rendimiento de manera interna sin requerir un conjunto adicional de validación.

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Generación de datos sintéticos
X, y = make_classification(n_samples=1000, n_features=20,
                          n_informative=15, n_redundant=5,
                          random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.3, random_state=42)

# Clasificador base: árbol de decisión
base_clf = DecisionTreeClassifier(random_state=42)

# Bagging (ajustado para versiones recientes de scikit-learn)
bagging_clf = BaggingClassifier(estimator=base_clf, # antes:
                               base_estimator
```

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

```
n_estimators=50,  
oob_score=True,  
random_state=42)  
bagging_clf.fit(X_train, y_train)  
  
print("Precisión en test:", accuracy_score(y_test,  
bagging_clf.predict(X_test)))  
print("Precisión OOB:", bagging_clf.oob_score_)
```

En el código presentado anteriormente, entrenamos un BaggingClassifier con 50 árboles de decisión. El modelo logró un **87.3% de precisión en test** y un **86.8% de precisión OOB**.

Esto nos muestra cómo Bagging aprovecha el muestreo con reemplazo para crear diversidad en los modelos y cómo la evaluación OOB nos da una estimación confiable del rendimiento sin necesidad de una validación cruzada externa. La cercanía de los valores confirma que el modelo está bien calibrado y no depende en exceso de los datos de entrenamiento.

1.2. Bosques Aleatorios (Random Forests) y Árboles Extremadamente Aleatorizados (Extra-Trees).

Los bosques aleatorios (Random Forests) representan una de las técnicas más populares y efectivas dentro de los métodos en conjunto.

Se basan en la idea de entrenar múltiples árboles de decisión utilizando diferentes subconjuntos de datos y subconjuntos de características, para luego combinar sus resultados mediante votación (en clasificación) o promedio (en regresión). Esta estrategia reduce significativamente la varianza del modelo y mitiga el riesgo de sobreajuste que presentan los árboles individuales (Ketkar, 2023).

La principal fortaleza de los Random Forests radica en el muestreo aleatorio de características en cada división del árbol. Esto fomenta la diversidad entre los modelos y, por ende, un ensamble más robusto.

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

Además, los bosques aleatorios ofrecen medidas de importancia de variables, lo cual resulta esencial en la interpretación de modelos en aplicaciones reales, como medicina, ingeniería o economía (Müller & Guido, 2022).

Por otro lado, los árboles extremadamente aleatorizados (Extra-Trees) llevan la aleatoriedad un paso más allá. Mientras que los Random Forests seleccionan el mejor umbral para dividir una característica en cada nodo, los Extra-Trees escogen umbrales de manera totalmente aleatoria. Esto produce árboles más “caóticos”, pero cuando se combinan, generan ensambles con alta capacidad de generalización y velocidad de entrenamiento superior, ya que se evita el cálculo exhaustivo de la mejor división (Amini & Bach, 2021).

Es importante señalar que los Random Forests priorizan el balance entre estabilidad y rendimiento, mientras que los Extra-Trees exploran la rapidez y la variabilidad extrema. Ambos enfoques permiten experimentar con la relación entre sesgo y varianza, lo cual constituye un eje central en el aprendizaje automático.

Figura 2. Comparación entre Bosques Aleatorios (Random Forests) y Árboles Extremadamente Aleatorizados (Extra-Trees)



Fuente: Elaboración propia

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

Con el propósito de comparar dos de los métodos en conjunto más utilizados en clasificación supervisada, se implementaron los algoritmos **Random Forest** y **Extra-Trees** empleando el dataset de cáncer de mama incluido en *scikit-learn*. Para ello, se dividieron los datos en conjuntos de entrenamiento y prueba mediante `train_test_split`, preservando un 30% para la evaluación. Ambos clasificadores se construyeron con 100 árboles (`n_estimators=100`) y una semilla fija de aleatoriedad para garantizar la reproducibilidad. El modelo de **Random Forest** entrena cada árbol a partir de subconjuntos de características seleccionadas aleatoriamente, mientras que **Extra-Trees** introduce un mayor grado de aleatoriedad al elegir los puntos de corte en las divisiones de los nodos. El rendimiento de ambos modelos se reporta con la función `classification_report`, que proporciona métricas de **precisión, recall, f1-score y exactitud global**, facilitando así un análisis comparativo de su desempeño en la tarea de clasificación.

```
from sklearn.ensemble import RandomForestClassifier,
ExtraTreesClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Dataset de cáncer de mama (scikit-learn)
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=42
)

# Random Forest
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X_train, y_train)
print("Random Forest:")
print(classification_report(y_test, rf_clf.predict(X_test)))

# Extra-Trees
et_clf = ExtraTreesClassifier(n_estimators=100, random_state=42)
et_clf.fit(X_train, y_train)
print("Extra-Trees:")
print(classification_report(y_test, et_clf.predict(X_test)))
```


Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

El modelo de **Random Forest** obtuvo un **accuracy del 97%**, lo que refleja un desempeño muy sólido en la clasificación del dataset de cáncer de mama. En detalle, la clase negativa (0) alcanzó una precisión del 98% y un recall del 94%, mientras que la clase positiva (1) mostró un recall del 99% y una precisión del 96%. Estos resultados indican que Random Forest presenta una gran capacidad para identificar correctamente los casos positivos, aunque con un pequeño margen de error en la detección de los negativos.

El modelo de **Extra-Trees** logró un rendimiento ligeramente superior, alcanzando un **accuracy del 98%**. La clase negativa (0) obtuvo una precisión del 98% y un recall del 95%, mientras que la clase positiva (1) mantuvo un recall del 99% y una precisión del 97%. Estas métricas reflejan un equilibrio aún mayor entre precisión y sensibilidad, reduciendo los falsos negativos y manteniendo la consistencia en los positivos.

Tanto **Random Forest** como **Extra-Trees** demostraron ser algoritmos altamente eficaces para problemas de clasificación supervisada. La diferencia esencial radica en que **Extra-Trees incorpora mayor aleatoriedad en la selección de las divisiones de los árboles**, lo que en este caso se tradujo en una ligera mejora en la capacidad de generalización y en la exactitud del modelo.

1.3. Métodos de Potenciación (Boosting: AdaBoost, Gradient Boosting, XGBoost).

A diferencia de bagging y bosques aleatorios, los métodos de **potenciación (boosting)** no entrenan modelos en paralelo, sino de forma **secuencial**.

Cada nuevo modelo busca corregir los errores cometidos por los anteriores, lo que permite construir un ensamble altamente preciso, aunque con mayor riesgo de sobreajuste si no se regula adecuadamente (Benítez et al., 2023).

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

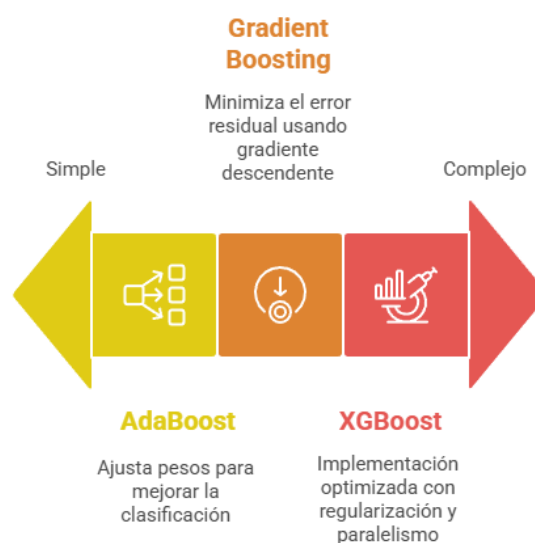
AdaBoost

El **Adaptive Boosting (AdaBoost)** fue uno de los primeros métodos de boosting. Su principio es sencillo: asignar más peso a las instancias mal clasificadas para que los clasificadores posteriores se concentren en ellas. Al final, combina los modelos débiles (generalmente árboles de decisión muy poco profundos, llamados *stumps*) en un clasificador fuerte. AdaBoost es particularmente útil cuando los datos presentan ruido moderado, ya que ajusta dinámicamente la importancia de cada observación (Amini & Bach, 2021).

Gradient Boosting

El **Gradient Boosting** da un paso más sofisticado: en lugar de ajustar pesos, entrena cada nuevo modelo para minimizar el error residual (la diferencia entre las predicciones actuales y los valores reales). Esto se logra utilizando el gradiente descendente sobre una función de pérdida, lo que lo convierte en un método flexible y altamente personalizable. El control de hiperparámetros como la tasa de aprendizaje (*learning rate*) y la profundidad de los árboles es clave para evitar sobreajuste (Müller & Guido, 2022).

Figura 3. Métodos de boosting clasificados por complejidad y optimización



Fuente: Elaboración propia

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

XGBoost

Finalmente, el **Extreme Gradient Boosting (XGBoost)** es una implementación optimizada del Gradient Boosting que introdujo mejoras en velocidad, regularización y manejo de valores faltantes. Se ha convertido en un estándar en competencias de Kaggle por su capacidad para lograr modelos de alto rendimiento en problemas tanto de clasificación como de regresión. Su eficiencia se basa en técnicas como el paralelismo, la poda temprana y el control fino de la complejidad del modelo (Chen & Guestrin, 2016; actualizado en implementaciones modernas como LightGBM y CatBoost).

Para profundizar en los métodos de *boosting*, se implementaron tres algoritmos representativos: **AdaBoost**, **Gradient Boosting** y **XGBoost**. Para ello, se generó un conjunto de datos sintéticos mediante `make_classification`, con 20 características (15 informativas y 5 redundantes), que posteriormente se dividieron en subconjuntos de entrenamiento y prueba con `train_test_split`. El modelo de **AdaBoostClassifier** se entrenó con 100 clasificadores débiles, actualizando iterativamente los pesos de las instancias mal clasificadas con el fin de mejorar la precisión en cada ronda. En el caso de **GradientBoostingClassifier**, se utilizó un aprendizaje aditivo basado en el gradiente, configurando un máximo de profundidad de 3 en los árboles y una tasa de aprendizaje de 0.1, lo que permite un control fino del ajuste del modelo. Finalmente, se incluyó **XGBoost**, una variante optimizada y ampliamente utilizada en competencias de ciencia de datos, que integra regularización y técnicas de optimización más eficientes. El rendimiento de cada modelo se evaluó con la métrica de **accuracy** sobre el conjunto de prueba, lo que facilita la comparación de sus capacidades predictivas en un mismo escenario experimental.

```
from sklearn.ensemble import AdaBoostClassifier,
GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Datos sintéticos
X, y = make_classification(n_samples=2000, n_features=20,
```

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

```
n_informative=15, n_redundant=5,
random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# AdaBoost
ada = AdaBoostClassifier(n_estimators=100, random_state=42)
ada.fit(X_train, y_train)
print("AdaBoost Accuracy:", accuracy_score(y_test,
ada.predict(X_test)))

# Gradient Boosting
gb = GradientBoostingClassifier(n_estimators=100,
learning_rate=0.1, max_depth=3, random_state=42)
gb.fit(X_train, y_train)
print("Gradient Boosting Accuracy:", accuracy_score(y_test,
gb.predict(X_test)))

# XGBoost
xgb = XGBClassifier(n_estimators=100, learning_rate=0.1,
max_depth=3, use_label_encoder=False, eval_metric='logloss')
xgb.fit(X_train, y_train)
print("XGBoost Accuracy:", accuracy_score(y_test,
xgb.predict(X_test)))
```

La implementación de los algoritmos de *boosting* sobre el conjunto de datos sintéticos mostró diferencias significativas en el rendimiento medido con la métrica de **accuracy**. El modelo de **AdaBoost** alcanzó una precisión del **81%**, evidenciando un desempeño aceptable, pero inferior en comparación con las otras variantes. Este resultado refleja la sensibilidad de AdaBoost frente a instancias ruidosas o difíciles de clasificar, dado que el algoritmo ajusta los pesos de los ejemplos mal clasificados en cada iteración, lo que puede conducir a cierto sobreajuste.

Por su parte, el modelo de **Gradient Boosting** obtuvo una precisión de **88.8%**, mostrando una mejora sustancial frente a AdaBoost. Este comportamiento se debe a su estrategia de optimización basada en el gradiente, que permite corregir los errores de manera más controlada mediante el uso de un *learning rate* y la limitación de la profundidad de los árboles, reduciendo así la varianza del modelo.

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

Finalmente, **XGBoost** alcanzó el mejor resultado con una precisión de **89.7%**, superando ligeramente a Gradient Boosting. Este rendimiento superior se explica por las optimizaciones que incorpora XGBoost, tales como la regularización explícita, la gestión eficiente de recursos computacionales y el manejo de interacciones no lineales con mayor robustez. Aunque el aumento de precisión frente a Gradient Boosting es marginal, su eficiencia en entrenamiento y su capacidad de escalar a grandes volúmenes de datos lo convierten en una herramienta ampliamente utilizada en aplicaciones reales y competencias de *data science*.

En conclusión, los resultados obtenidos reflejan la evolución progresiva de los algoritmos de *boosting*: AdaBoost como un método fundacional y más sensible al ruido, Gradient Boosting como una versión más estable y precisa, y XGBoost como una optimización de alto rendimiento tanto en exactitud como en eficiencia computacional.

1.4. Stacking y Blending de Modelos.

Los métodos de **stacking** y **blending** representan un nivel más avanzado dentro de los modelos de ensamble, ya que no solo combinan predicciones, sino que aprenden a combinar modelos.

En lugar de limitarse a promediar o votar como en bagging o boosting, estos enfoques entrenan un modelo adicional —llamado **meta-modelo**— que aprende a integrar las predicciones de múltiples modelos base (Wolpert, 1992; actualizado en aplicaciones recientes por Müller & Guido, 2022).

En el **stacking (stacked generalization)**, los modelos de nivel 0 (árboles, regresiones, SVM, redes neuronales, etc.) realizan predicciones que luego se usan como entradas para un modelo de nivel 1, el meta-modelo. Este meta-modelo aprende qué combinaciones son más fiables para reducir el error de generalización. Por ejemplo, un Random Forest puede capturar interacciones no lineales, una regresión logística puede aportar interpretabilidad, y un SVM puede ofrecer precisión en fronteras

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

complejas. La combinación estratégica de todos estos modelos permite robustecer la predicción final (Amini & Bach, 2021).

Por otro lado, el **blending** sigue un principio similar, pero en lugar de utilizar validación cruzada para generar las predicciones de entrenamiento para el meta-modelo, emplea un conjunto de validación separado. Esto simplifica el proceso, aunque con el riesgo de aprovechar menos datos para el aprendizaje. En entornos competitivos como Kaggle, blending se utiliza frecuentemente como un atajo práctico para mejorar resultados rápidamente.

Figura 4. Comparación de Stacking y Blending en Modelos de Ensamble



Fuente: Elaboración propia

Ningún algoritmo es universalmente el mejor (teorema de “no free lunch”). La clave está en reconocer que distintos algoritmos capturan diferentes patrones y, al combinarse, pueden ofrecer un desempeño superior al de cualquier modelo individual. Con el objetivo de ejemplificar el funcionamiento del método de **Stacking**, se empleó el conjunto de

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

datos de vino incluido en *scikit-learn*. En este caso, se combinaron como modelos base un **Random Forest**, un **Gradient Boosting** y una **Máquina de Vectores de Soporte (SVM)**, cuya diversidad permite capturar diferentes patrones en los datos. Sobre sus predicciones se entrenó un meta-modelo de **regresión logística**, encargado de integrar los resultados parciales en una decisión final más robusta. La implementación, realizada con la clase `StackingClassifier`, incluye validación cruzada de cinco particiones (`cv=5`) para mejorar la estimación del rendimiento. Finalmente, se calcula la precisión del modelo sobre el conjunto de prueba, mostrando cómo la estrategia de apilar clasificadores puede incrementar la capacidad predictiva frente a la utilización de un único algoritmo de manera aislada.

```
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Dataset: Vino
data = load_wine()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=42
)

# Modelos base
rf = RandomForestClassifier(n_estimators=100, random_state=42)
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
svm = SVC(probability=True, random_state=42)

estimators = [
    ('rf', rf),
    ('gb', gb),
    ('svm', svm)
]

# Meta-modelo (Stacking)
stacking_clf = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(max_iter=1000),
    cv=5
)
```


Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

```
# Entrenamiento de cada modelo
rf.fit(X_train, y_train)
gb.fit(X_train, y_train)
svm.fit(X_train, y_train)
stacking_clf.fit(X_train, y_train)

# Evaluación en el conjunto de prueba
print("Precisión Random Forest:", accuracy_score(y_test,
rf.predict(X_test)))
print("Precisión Gradient Boosting:", accuracy_score(y_test,
gb.predict(X_test)))
print("Precisión SVM:", accuracy_score(y_test,
svm.predict(X_test)))
print("Precisión con Stacking:", accuracy_score(y_test,
stacking_clf.predict(X_test)))
```

La evaluación de los clasificadores sobre el dataset de vino muestra diferencias notables en el rendimiento de cada modelo. El **Random Forest** alcanzó una **precisión perfecta (100%)**, lo que indica un ajuste excelente a este conjunto de datos, aunque también podría sugerir un posible sobreajuste que convendría verificar en otros escenarios. El **Gradient Boosting** obtuvo una precisión del **90.7%**, mostrando un buen desempeño, pero inferior al de Random Forest. Por su parte, la **SVM** alcanzó únicamente un **75.9%**, evidenciando mayores dificultades para capturar la complejidad de los datos en comparación con los otros algoritmos.

En cuanto al **Stacking**, la precisión fue del **98.1%**, superando claramente a Gradient Boosting y SVM, y acercándose al rendimiento del Random Forest. Esto refleja cómo la combinación de diferentes modelos mediante un meta-aprendizaje basado en **regresión logística** logra integrar las fortalezas de cada clasificador y compensar sus debilidades individuales. En términos pedagógicos, este resultado evidencia que el **Stacking no siempre supera al mejor modelo individual**, pero ofrece una solución más **robusta y generalizable**, lo que lo convierte en una estrategia atractiva en contextos donde no se conoce de antemano qué clasificador será el más efectivo.

Bagging

Bosques Aleatorios

Métodos Potenciación

Stacking y Blending

Bibliografía

Amini, M. R., & Bach, F. (2021). Apprentissage machine: De la théorie à la pratique. Paris: Eyrolles.

Benítez, R., Cencerrado Barraqué, A., Escudero, G., & Kanaan, S. (2023). Inteligencia artificial avanzada. Universitat Oberta de Catalunya (UOC).

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. <https://doi.org/10.1145/2939672.2939785>

Ketkar, N. (2023). Deep Learning with Python: A hands-on introduction. Springer.

Müller, A. C., & Guido, S. (2022). Introduction to Machine Learning with Python: A guide for data scientists. O'Reilly Media.

Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241–259. [https://doi.org/10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1)